MariaDB

Control Structures

Estructuras de Control en MariaDB

Automatiza tareas y gestiona datos eficientemente con las estructuras de control del servidor SQL

SQL

```
IF (condición) THEN
    -- instrucciones
ELSE  -- otras instrucciones
END IF;
```

IF...THEN...ELSE | CASE | WHILE | REPEAT...UNTIL | CURSOR

Estructura Condicional: IF...THEN...ELSE

¿Qué es?

Permite ejecutar una acción si se cumple una condición, y otra si no se cumple.



¿Por qué es útil?

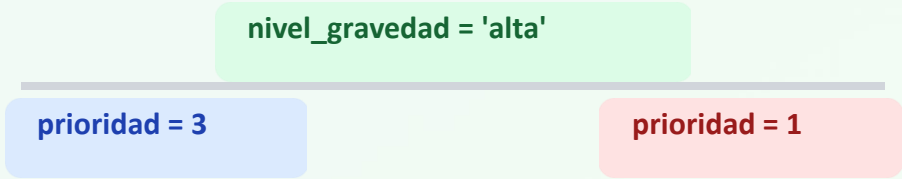
Muchas veces necesitamos adaptar el comportamiento según los datos. Por ejemplo, si una incidencia es grave, queremos tratarla con más urgencia.

Sintaxis

```
IF condición THEN
    -- instrucciones si se cumple
ELSE
    -- instrucciones si no se cumple
END IF;
```

Ejemplo

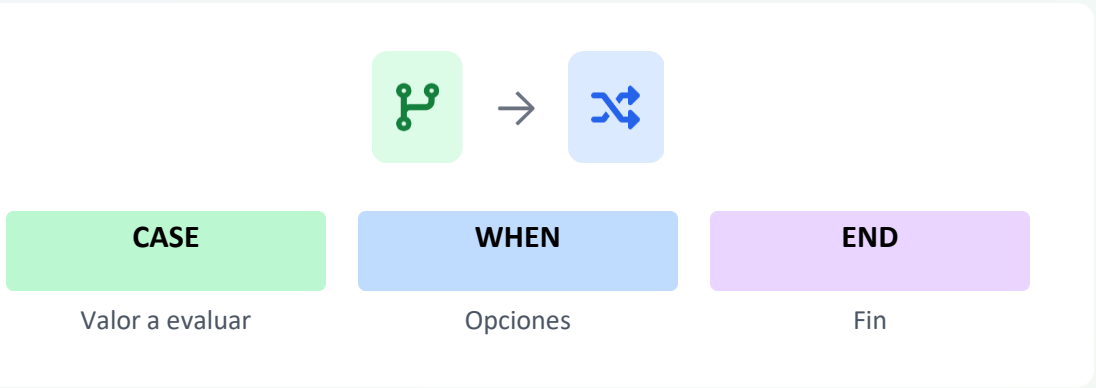
```
IF nivel_gravedad = 'alta' THEN
    SET prioridad = 3;
ELSE
    SET prioridad = 1;
END IF;
```



Estructura Condicional Múltiple: CASE

¿Qué es?

Es como un "IF" con varias opciones. Muy útil para clasificar o transformar transformar valores.



¿Por qué es útil?

Permite escribir código más limpio cuando hay varias condiciones posibles.

Sintaxis

```
CASE valor
    WHEN opción1 THEN resultado1
    WHEN opción2 THEN resultado2
    ELSE resultado_por_defecto
END;
```

Ejemplo

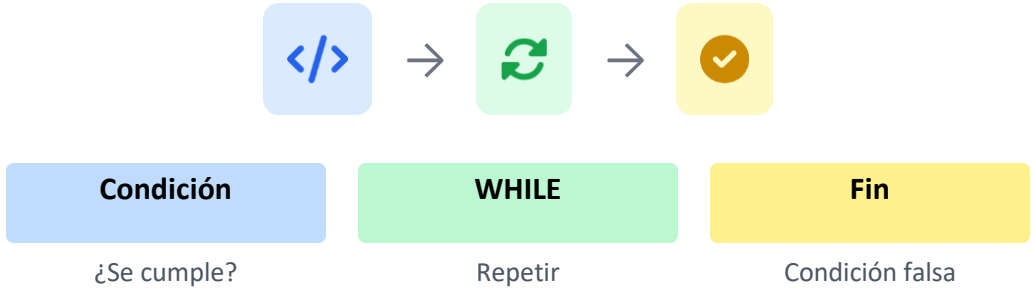
```
SET prioridad = CASE nivel_gravedad
    WHEN 'baja' THEN 1
    WHEN 'media' THEN 2
    WHEN 'alta' THEN 3
    ELSE 0
END;
```



Bucle WHILE

¿Qué es?

Permite repetir una acción mientras se cumpla una condición.



¿Por qué es útil?

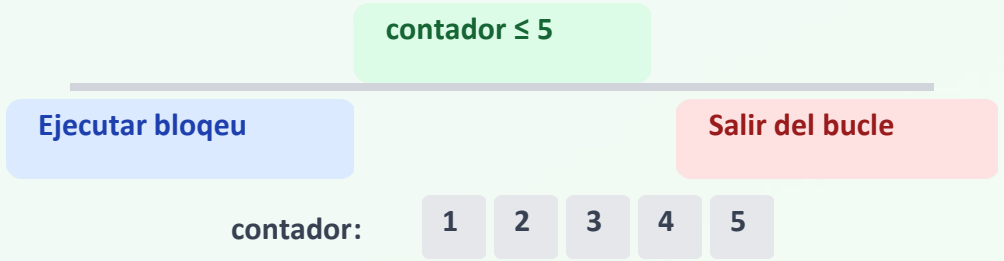
A veces necesitamos recorrer datos o repetir tareas, como revisar varias incidencias o generar informes.

Sintaxis

```
WHILE condición DO
    -- instrucciones
END WHILE;
```

Ejemplo

```
SET contador = 1;
WHILE contador <= 5 DO
    -- aquí podríamos insertar registros, mostrar mensajes, etc.
    SET contador = contador + 1;
END WHILE;
```



Bucle REPEAT...UNTIL

¿Qué es?

Repite las instrucciones al menos una vez, y luego verifica si debe continuar.



¿Por qué es útil?

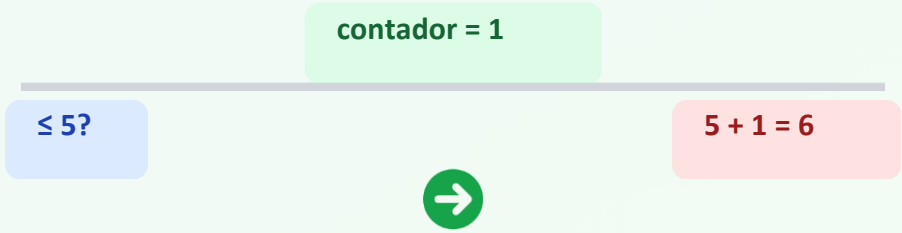
Porque garantiza que el bloque se ejecuta al menos una vez, útil cuando queremos hacer una primera acción antes de verificar.

Sintaxis

```
REPEAT
    -- instrucciones
UNTIL condición_de_salida
END REPEAT;
```

Ejemplo

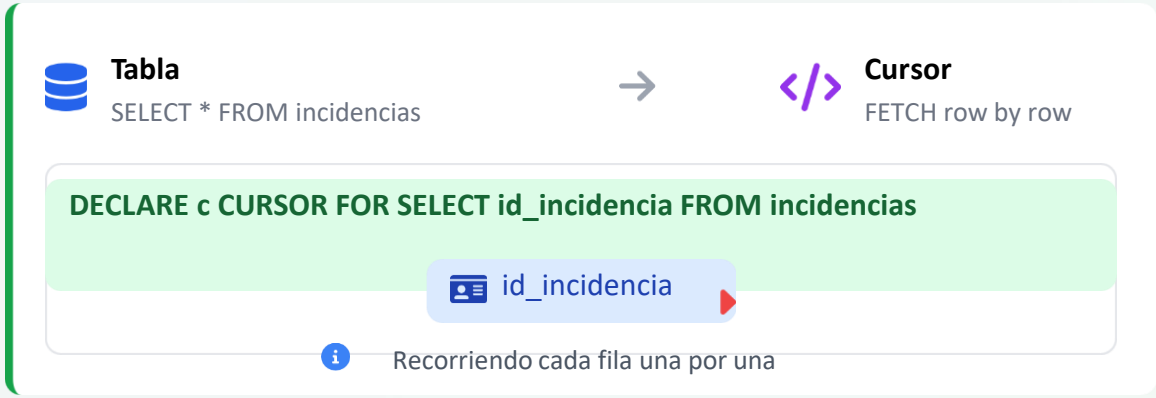
```
SET contador = 1;
REPEAT
    SET contador = contador + 1;
UNTIL contador > 5
END REPEAT;
```



Cursores en MariaDB

¿Qué es un cursor?

Un cursor permite recorrer fila por fila el resultado de una consulta.



¿Por qué es útil?

Permite procesar registros uno a uno, ideal para tareas administrativas como enviar alertas, generar informes o aplicar reglas.

Sintaxis básica

```
DECLARE nombre_cursor CURSOR FOR
  SELECT ...;
OPEN nombre_cursor;
FETCH nombre_cursor INTO variable1, variable2;
CLOSE nombre_cursor;
```

Ejemplo

```
DECLARE c CURSOR FOR SELECT id_incidencia FROM incidencias;
DECLARE done INT DEFAULT FALSE;
DECLARE id_actual INT;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
OPEN c;
REPEAT
  FETCH c INTO id_actual;
  IF NOT done THEN
    -- hacer algo con id_actual
END IF;
UNTIL done
END REPEAT;
CLOSE c;
```

Cursores en MariaDB

¿Qué es un cursor?

Un cursor es un mecanismo que utilizamos dentro de procedimientos y estructuras de control para almacenar información de cada fila en variables y operar con ellas fila por fila, de tal manera que el resultado se muestra, fila por fila y las variables van cambiando su valor en función de cómo vamos recorriendo registros en esa consulta. Una vez que ya he hecho el recorrido de todas las filas, el cursor ya no se necesita y se elimina el cursor.

Las Variables Locales (DECLARE v_nombre...): Viven durante toda la ejecución del procedimiento almacenado (desde el BEGIN hasta el END). No se borran al cerrar el cursor, simplemente se quedan con el último valor que leyeron en la última fila.

Explicación del Ejemplo

1. Declaración del cursor
2. Declaración de la variable “done” (terminado) de tipo entero y a la que se asigna el valor inicial FALSE (que MySQL lo guarda automáticamente como 0).
3. Declaración de la variable id_actual, que es la variable que guarda el valor recogido de cada fila en la que se vaya encontrando el cursor cada vez, en cada vuelta de bucle.
4. La sentencia “DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;” sirve para decirle al sistema que si en algún momento, cuando ejecute “FETCH” (leer una fila de la tabla), no encuentra más, que asigne el valor TRUE a la variable done para que el cursor se pare.
5. Abrir el cursor
6. Iniciar el bucle REPEAT
7. Lectura de la fila actual SI la condición de done=false
8. Se ejecuta la “tarea que se quiera” en cada fila
9. Se cierra la condición
10. Se cierra el bucle si done = TRUE
11. Dejamos de repetir
12. Cerramos el cursor

Sintaxis básica

```
DECLARE nombre_cursor CURSOR FOR
    SELECT ...;

OPEN nombre_cursor;

FETCH nombre_cursor INTO variable1, variable2;

CLOSE nombre_cursor;
```

Ejemplo

```
DECLARE c CURSOR FOR SELECT id_incidencia FROM incidencias;
DECLARE done INT DEFAULT FALSE;
DECLARE id_actual INT;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

OPEN c;
REPEAT
    FETCH c INTO id_actual;
    IF NOT done THEN
        -- hacer algo con id_actual
END IF;
UNTIL done
END REPEAT;
CLOSE c;
```

Conclusiones

⭐ Lo esencial

Las estructuras de control en MariaDB no son para programadores, sino para sino para administradores que quieren automatizar tareas dentro de la base de la base de datos.

Herramientas poderosas

Permiten tomar decisiones, repetir acciones y recorrer datos, todo todo desde el propio servidor SQL.

Automatización efectiva

Facilitan la ejecución de tareas administrativas complejas sin necesidad de scripts externos.

Gestión de datos

Aprenderlas no es aprender a programar, es aprender a gestionar mejor los datos.

Estructuras de Control

IF...THEN...ELSE | CASE | WHILE | REPEAT...UNTIL | CURS



> *Las estructuras de control te permiten transformar tu base de datos en una herramienta automatizada y poderosa.*